

Padrões de Projeto Criacional	
PANDORA - Plataforma Inteligente de Locação e Manutenção de Equipamentos com LLM	

Padrões de projeto criacionais do PANDORA

E2-04 | Linguagem de Programação III | 2ª fase do Projeto Interdisciplinar IV

O backend Django do PANDORA contém quatro padrões criacionais. Este documento apresenta problema, justificativa, diagramas originais dos artefatos e implementação funcional, indicando as integrações pendentes.

Equipe: Eduardo Souza Gomes (116758); Felipe Antonio Parolin (116276); Gabriel Eduardo da Cunha (116849); Gabriel Moi Stensen (116743); Matheus Henrique Araujo Silva (117224); Rafael Donizete Mantoan (116305).

1 Aplicações e validação

Padrão	Aplicação atual	Justificativa
Singleton	AISettings e AIService	Compartilhar a configuração de IA por processo.
Builder	Abertura de OS pela API	Centralizar a montagem e os campos mínimos da ordem.
Factory Method	Processadores de relatos	Separar a criação do produto do fluxo de processamento.
Abstract Factory	Checklist e roteiro de manutenção	Produzir elementos compatíveis da mesma modalidade.

Verificação em 03/10/2026, commit 071a9fe7f5006497d32f9b71e0c7302e0f3b206b: os sete testes de core e maintenance passaram com Django 6.0.4 e SQLite temporário. As chamadas de texto e dos dois kits de manutenção também funcionaram. No Builder, verificações adicionais confirmaram criação pela API (HTTP 201), campo ausente sem persistência e relato vazio rejeitado (HTTP 400). O recorte não substitui a validação em PostgreSQL.

Reprodução. No diretório api, instalar requirements.txt, definir DB_ENGINE=sqlite no ambiente e executar:

```
python manage.py test core.tests maintenance.tests
```

1 1 Código completo e fontes

[Singleton — implementação e consumidor](#)

[Builder, Factory Method e Abstract Factory — implementação](#)

[Builder — integração no serializer da API](#)

[Abstract Factory — endpoint de modelos de checklist](#)

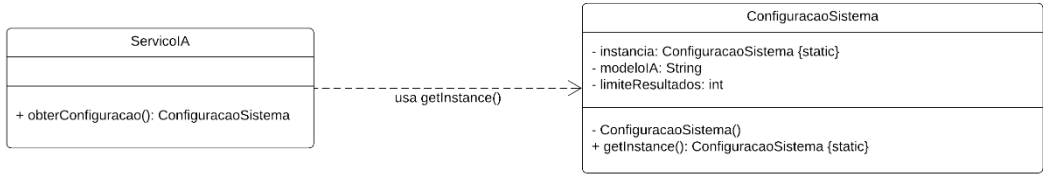
[Testes dos padrões de manutenção](#)

Base documental: Molde.docx; backlog E2-04; aulas de LP III; Modelagem dos Padrões (Preliminar); Atividade MA1. Os links apontam para o código completo no commit verificado; os exemplos resumem seu uso.

2 Singleton

Problema e justificativa. Diferentes componentes podem carregar configurações independentes de IA. O Singleton centraliza modelo e limite de resultados, oferecendo uma referência compartilhada. Seu uso é restrito à configuração, evitando armazenar dados mutáveis de usuários em estado global.

PANDORA - 1. Singleton



{Uma única instância por processo; inicialização protegida contra concorrência.
getInstance() cria na primeira chamada e reutiliza nas seguintes.

Figura 1 Singleton da modelagem existente

Correspondência com o código. ConfiguracaoSistema corresponde a AISettings; ServicoIA corresponde a AIService. O acesso modelado como getInstance() é implementado por get_instance().

Implementação. Em api/core/patterns.py, AISettings é uma dataclass imutável. get_instance() mantém a instância em _instance e usa Lock na inicialização. AIService recebe essa configuração por injeção ou usa a instância compartilhada.

```
@classmethod
def get_instance(cls):
    if cls._instance is None:
        with cls._lock:
            if cls._instance is None:
                cls._instance = cls(
                    model=getattr(
                        settings, "AI_MODEL", "llama-3.3-70b-versatile"
                    ),
                    result_limit=getattr(settings, "AI_RESULT_LIMIT", 5),
                )
    return cls._instance
```

Exemplo funcional. A chamada abaixo retornou True; o teste existente também confirma que instance() e get_instance() devolvem a mesma referência.

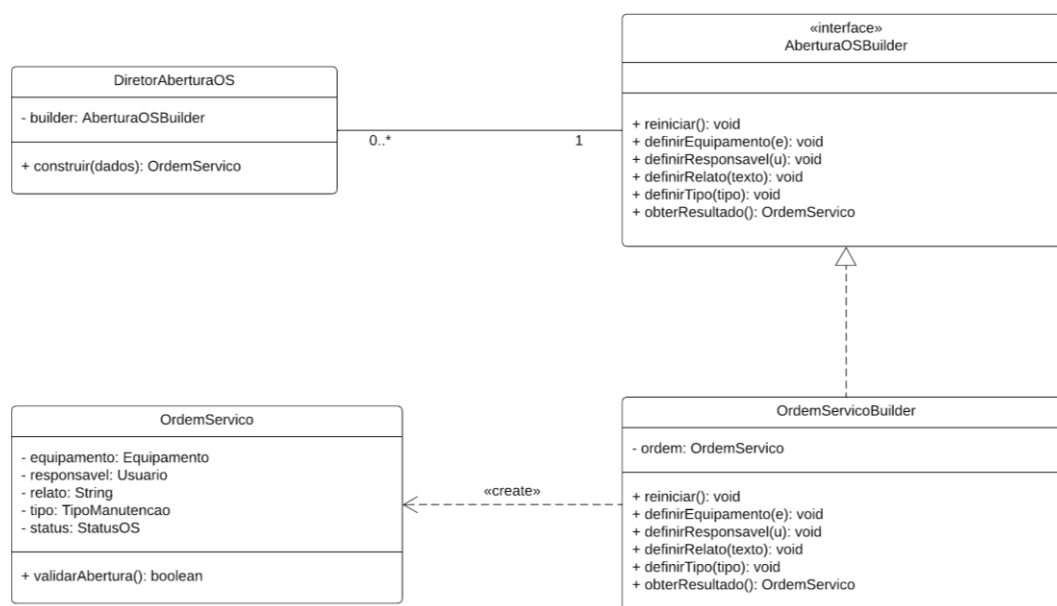
```
AISettings.get_instance() is AIService().get_configuration()
# Resultado: True
```

Limites e impacto. A unicidade vale por processo e pelo acesso a get_instance(); Python permite construir AISettings diretamente, diferentemente do construtor privado desenhado. AIService é um consumidor de configuração, sem integração de LLM demonstrada nesse trecho. O padrão reduz duplicação, mas adiciona estado compartilhado e não seria necessário se a configuração nativa do Django bastasse.

3 Builder

Problema e justificativa. A abertura da OS reúne equipamento, usuário, sintomas, tipo e campos opcionais. O Builder centraliza a montagem em etapas e exige os campos mínimos antes de persistir; o Director organiza a sequência.

PANDORA - 2. Builder



O diretor coordena as etapas; o builder concreto monta uma nova OS e valida a abertura. Atividades executadas e teste final pertencem às etapas posteriores da manutenção.

Figura 2 Builder da modelagem existente

Correspondência com o código. AberturaOSBuilder → ServiceOrderBuilderContract; OrdemServicoBuilder → ServiceOrderBuilder; DiretorAberturaOS → ServiceOrderOpeningDirector; OrdemServico → ServiceOrder. O contrato é uma classe abstrata Python.

Implementação e uso real. ServiceOrderSerializer.create() chama o Director em POST /api/service-orders/. Ele executa reset(), equipment(), responsible(), report(), kind() e build(). Exemplo, com equipamento e usuário existentes:

```

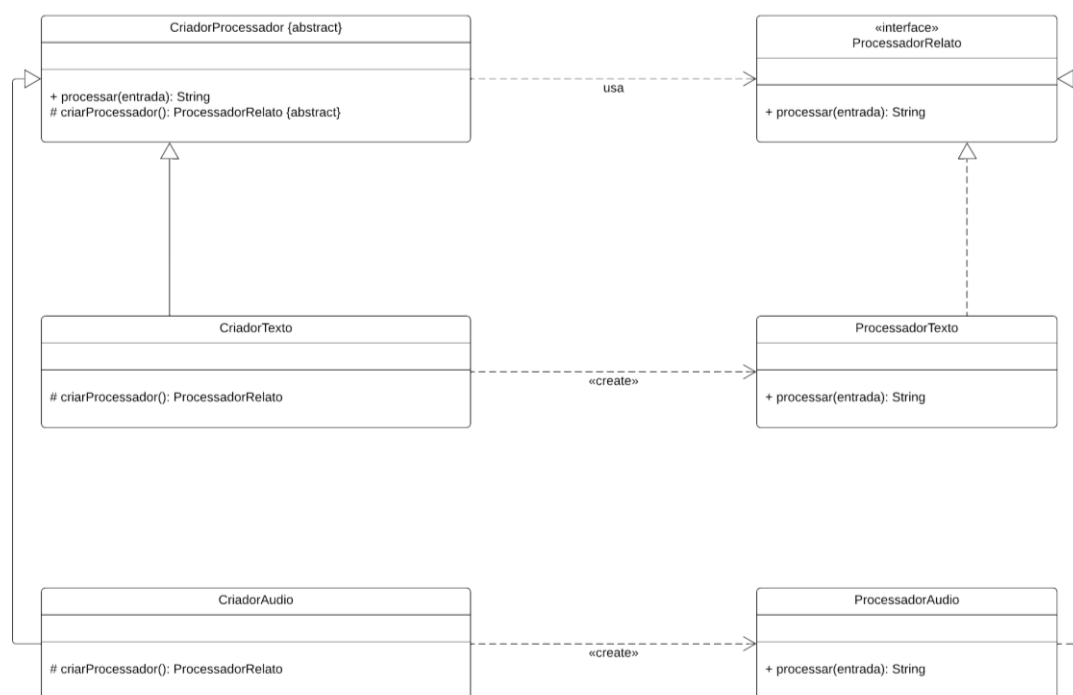
order = ServiceOrderOpeningDirector().open(
    equipment=equipment, opened_by=user,
    symptoms="Falha no motor", maintenance_type="CORRECTIVE"
)
  
```

Limites e evidência. _data acumula os campos; build() exige equipment, opened_by, symptoms e maintenance_type, persiste e limpa o estado após sucesso. A OS só existe ao final, diferentemente do desenho. A presença das chaves não valida valores; build() não chama full_clean(). O serializer valida a entrada e usa request.user. A API criou a OS com HTTP 201; o padrão não realiza diagnóstico ou liberação técnica.

4 Factory Method

Problema e justificativa. Texto e áudio exigem processadores distintos. ReportProcessorCreator mantém o fluxo comum; os creators concretos criam produtos pelo contrato ReportProcessor, reduzindo o acoplamento com cada formato.

PANDORA - 3. Factory Method



processar() chama criarProcessador() e delega o tratamento ao produto retornado.
 Texto: normaliza o relato. Áudio: transcreve e normaliza; a saída segue para revisão humana.

Figura 3 Factory Method da modelagem existente

Correspondência com o código. CriadorProcessador → ReportProcessorCreator; CriadorTexto e CriadorAudio → TextReportProcessorCreator e AudioReportProcessorCreator; ProcessadorRelato → ReportProcessor. Os produtos são TextReportProcessor e AudioReportProcessor.

Implementação funcional. process() chama create_processor().process(content); cada creator sobrescreve create_processor(). ReportProcessorFactory seleciona e delega ao creator. O padrão está nessa hierarquia, não apenas no seletor. Chamada verificada:

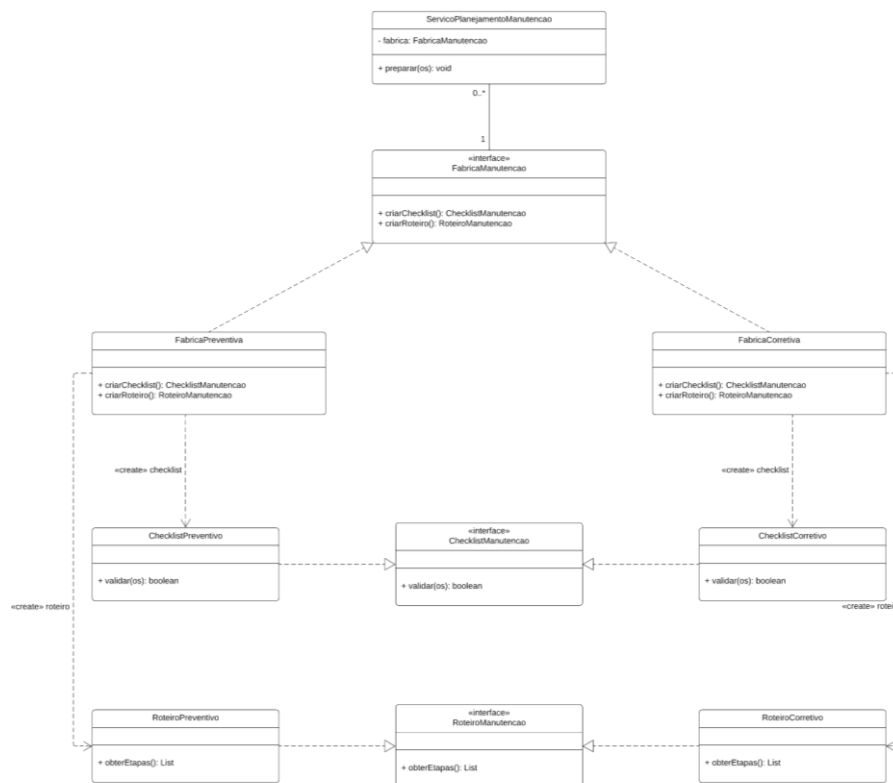
```
ReportProcessorFactory.process("text", " Falha no motor ")
# Resultado: "Falha no motor"
```

Limites e impacto. Texto normaliza espaços. Áudio tem creator e produto, mas process() lança NotImplementedException: transcrição pendente, apesar da previsão no diagrama. O serializer de OS ainda não chama esses processadores. Acrescentar formatos fica mais simples, ao custo de mais classes; o fluxo de voz ainda não funciona.

5 Abstract Factory

Problema e justificativa. Checklist e roteiro precisam formar a mesma modalidade. Abstract Factory cria famílias preventiva e corretiva pelos contratos MaintenanceChecklist e MaintenanceRoute, sem duplicar ServiceOrder.

PANDORA - 4. Abstract Factory



Cada fábrica produz checklist e roteiro da mesma modalidade de manutenção.
Proposta aplicável se as modalidades exigirem comportamentos distintos; a entidade OrdemService permanece única.

Figura 4 Abstract Factory da modelagem existente

Correspondência com o código. FabricaManutencao → MaintenanceFamilyFactory; FabricaPreventiva/Corretiva → PreventiveMaintenanceFactory/CorrectiveMaintenanceFactory; ServicoPlanejamentoManutencao → MaintenancePlanningService. Os produtos concretos são PreventiveChecklist, CorrectiveChecklist, PreventiveRoute e CorrectiveRoute.

Implementação funcional. prepare() chama create_checklist() e create_route(), reunindo items() e steps() em MaintenanceKit. Exemplo verificado:

```

kit = MaintenancePlanningService(
    PreventiveMaintenanceFactory()
).prepare()
  
```

Uso e limites. MaintenanceKitFactory: PREVENTIVE seleciona a preventiva; outros valores, a corretiva. GET /api/service-orders/checklist-template/?type=PREVENTIVE retorna checklist e route. O validar(os) desenhado não existe; o kit fornece dados, sem inspecionar ou liberar equipamentos. Hoje, os produtos contêm dados fixos; configuração é uma alternativa mais simples até exigirem comportamentos próprios.